ST ALOYSIUS
(DEEMED TO BE UNIVERSITY)
MANGALURU 575 003 - INDIA
SCHOOL OF ENGINEERING

Multi-Process Microcontroller Simulator: Design, Implementation & Evaluation of STC89C52 Microcontroller

Register Number	Student Name
25190111	CYRUS SHOBITH PINTO
25190110	CHRISEL LOBO
25190106	ARNOLD NOAH KARYAPPA
25190148	SHASHIDHARA POOJARI

Department of Computer Science & Engineering

Project-Based Learning (PBL) Report

ABSTRACT

This report presents the design, implementation, and empirical evaluation of an educational, instruction-accurate multi-process software simulator for a 19-opcode subset of the STC89C52 (Intel MCS-51) 8-bit microcontroller [1], [2]. Many educational microcontroller simulators use single-threaded monolithic architectures where user interface rendering or synchronous disk logging can stall the simulated CPU loop [3]. To eliminate this coupling, we engineered a three-process system communicating over local TCP loopback sockets: CoreProcess (CPU Fetch-Decode-Execute engine, 8 KB code ROM, 256-byte RAM, hardware stack, and 16-slot circular FIFO queue on port 5000), LoggerProcess (append-only execution log daemon writing to simulator.log on port 5001), and UIProcess (HTTP web server and 7-panel interactive dashboard on port 8080). Additionally, we implemented the 3GPP TS 36.212 CRC-24b error-detection polynomial (g(D) = 0x800063) [5] as a Java routine operating on simulated 8-bit registers R5, R6, and R7. Across 38 automated test assertions—including an exhaustive sweep of all 96 single-bit and 4,560 two-bit codeword errors—the system achieved 100% verification pass rates while maintaining sub-millisecond median interactive step latency (0.914 ms p50).

Keywords: Microcontroller Emulation, Multi-Process Architecture, Inter-Process Communication, Fetch-Decode-Execute, Instruction-Accurate Virtualization, CRC-24b Error Detection.

TABLE OF CONTENTS

1. Introduction	4
1.1 Problem Statement	4
1.2 Project Objectives	4
2. The Simulator Design and Working	4
2.1 Microcontroller Architecture Study	4
2.2 System Architecture and Design Decisions	4
2.3 Operational Workflow and GUI Layout	4
2.4 Incremental Implementation by Week	5
2.5 Multi-Process IPC Architecture & Benchmarking	5
3. Simulation Examples Testing and Screenshots	5
4. Individual Contribution	6
4.1 Weekly Task Breakdown by Member	6
4.2 Agreed Team Contribution Summary	6
4.3 Individual Personal Reflections	6
5. Team Communication	7
6. Problems Faced and How They Were Resolved	7
7. From Theory to Practice	7
7.1 Operating Systems Concepts	7
7.2 Data Structures & Algorithms	7
7.3 Computer Networks	7
7.4 Cross-Disciplinary Synthesis Reflection	7
8. Conclusion, Limitations and Future Work	8
References	8
Final Report Sign-off & Faculty Mentor Endorsement	9

1. Introduction

1.1 Problem Statement

Modern computer engineering pedagogy requires students to understand how hardware state machines, memory segmentation, and instruction pipelines execute in silicon [1]. However, physical kits present maintenance bottlenecks and lack visibility into internal registers. Furthermore, many educational simulators [3] use monolithic, single-threaded designs where GUI rendering or synchronous disk logging can stall the simulated CPU loop. Modeling telecommunications error-detection algorithms—such as 3GPP LTE CRC-24b [5]—across an 8-bit register file also requires multi-byte register cascading (R5:R6:R7) and shifting through the carry flag in GF(2) arithmetic.

1.2 Project Objectives

1. Virtualize a 19-opcode educational subset of the STC89C52/8051 [2] in Java with instruction-accurate Fetch-Decode-Execute stages, registers R0–R7, and PSW flags (CY, AC, OV, P).
2. Implement a three-process architecture over TCP loopback sockets (ports 5000, 5001, 8080) with a bounded non-blocking ArrayBlockingQueue(1024) log buffer prioritizing CPU determinism.
3. Construct a Web Dashboard (Port 8080, Figure 2) featuring live disassembly, stack visualization, and 16-slot circular FIFO queue inspection.
4. Build an independent append-only logging daemon on Port 5001 writing execution traces to simulator.log.
5. Implement and benchmark 3GPP TS 36.212 CRC-24b [5] as a Java routine operating on simulated registers R5:R6:R7 (Figure 3), verifying all 96 single-bit and 4,560 two-bit codeword error syndromes.

2. The Simulator Design and Working

2.1 Microcontroller Architecture Study

The target MCU (STC89C52RC / Intel MCS-51 [2]) possesses a hardware hierarchy mapped within our 19-opcode educational virtualization environment as summarized in Table 1.

Table 1: *STC89C52 / MCS-51 Hardware Specification [2] and Simulator Virtualization Design*

Hardware Feature	MCS-51 Hardware Specification [2]	Simulator Virtualization Design
Data Word Size	8-bit ALU & Datapath	Bitwise mask (& 0xFF) on all registers and RAM cells to prevent Java 32-bit sign-extension.
Program Counter (PC)	16-bit addressing (8 KB On-Chip ROM)	16-bit unsigned index into 8,192-byte ROM; halts on synthetic 0xFF or 8051 SJMP $ (80 FE).
Internal RAM	256 Bytes on-chip (00H - FFH)	Partitioned into Register Banks (00H-1FH), Bit-Addressable RAM (20H-2FH), Scratchpad (30H-7FH).
Special Function Regs	Direct addressing (80H - FFH)	Mapped SFRs: Accumulator (0xE0), B (0xF0), PSW (0xD0), Stack Pointer (0x81), Ports P0-P3.
ALU Status Flags	Program Status Word (PSW)	Boolean flags tracking Carry (CY), Auxiliary Carry (AC), Overflow (OV), and even Parity (P).
Stack & FIFO Queue	Hardware SP + 16-Slot Buffer	Pre-increment PUSH / post-decrement POP (SP=07H reset) + 16-slot circular FIFO ring buffer.

2.2 System Architecture and Design Decisions

The simulator executes across three OS processes over Java TCP loopback sockets (Figure 1, Panel A): CoreProcess (19-opcode CPU engine, 8 KB ROM, 256B RAM, 16-slot FIFO queue, and CRC-24b routine on port 5000), LoggerProcess (append-only logger writing to simulator.log on port 5001), and UIProcess (HTTP web server and CLI controller on port 8080). To ensure a stalled LoggerProcess never blocks CPU steps when OS socket buffers fill, CoreProcess enqueues log frames into a bounded 1,024-slot ArrayBlockingQueue via non-blocking offer() with an atomic drop counter (trading log completeness under extreme backpressure for guaranteed CPU determinism).

2.3 Operational Workflow and GUI Layout

As shown in the Figure 1 (Panel B) workflow flowchart and Figure 2 dashboard screenshot, http://localhost:8080/ exposes 7 panels: (1) Program Memory & Disassembly with PC pointer, (2) Fetch-Decode-Execute Pipeline Indicator, (3) Working Registers & PSW Flags, (4) Hardware LIFO Stack Tower, (5) 16-Slot Circular Queue Grid, (6) 16x16 Internal RAM Hex Matrix, and (7) the 3GPP LTE-B CRC-24b Lab (Figure 3). Browser actions invoke GET /api/cmd?cmd=... which UIProcess forwards over TCP Port 5000 to CoreProcess, returning STATE|... snapshots.

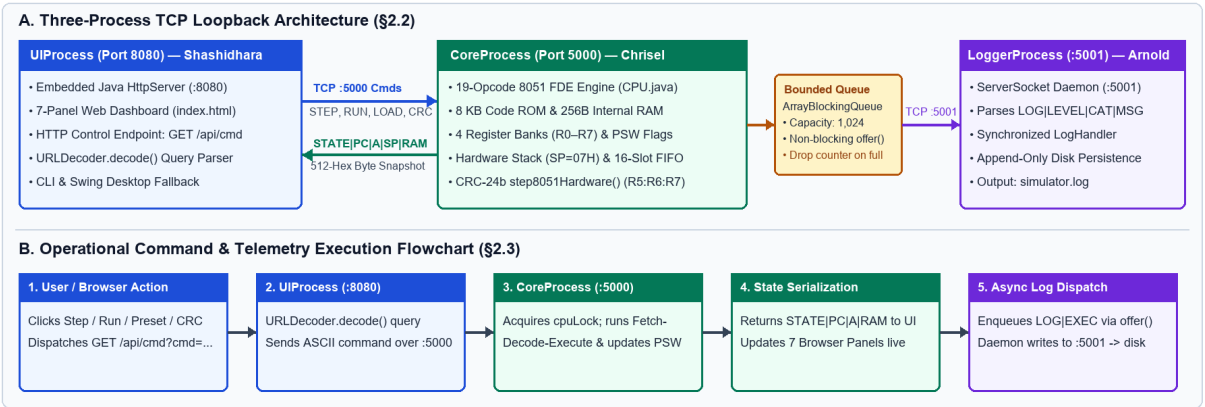


Figure 1: *(A) Three-Process TCP Loopback Architecture with Bounded 1,024-Slot Log Queue (§2.2) and (B) Operational Command & Telemetry Execution Flowchart (§2.3).*

2.4 Incremental Implementation by Week

Table 2: Five-Week Incremental Implementation Schedule and Deliverables [1]

Sprint / Week	Primary Module Focus	Concrete Functional Deliverables
Week 1	Architecture Modeling & Tooling	8051 register model, 8 KB ROM / 256B RAM map, Java project structure, and Git workflow.
Week 2	Core ALU & Instruction Subset	Fetch-Decode-Execute pipeline, 19 opcodes (MOV, ADD, SUBB, MUL, DIV, ANL, ORL, XRL, JZ, JNZ, SJMP), PSW.
Week 3	Data Structures & Memory	Hardware stack pointer (SP=07H reset), 16-slot circular FIFO queue, and QueueTest.java.
Week 4	Multi-Process Socket IPC	Decoupled Core (5000), Logger (5001), UI (8080), and bounded 1,024-slot non-blocking log queue.
Week 5	Telecom Extension & Benchmarks	3GPP LTE-B CRC-24b Java routine (step8051Hardware on R5:R6:R7), 4,656-case sweep, BenchmarkCRC.java.

2.5 Multi-Process IPC Architecture & Benchmarking

BenchmarkCRC.java was executed on Windows 11 x64 (OpenJDK 26.0.2) after a 10,000-iteration C2 JIT warm-up on a 49-byte telecom payload: (1) Standalone in-memory execution completed 100,000 CRC-24b iterations in 28.92 ms (0.0003 ms/pkt, ~3.46M pkts/sec, checksum 0x025A3C); (2) Multi-process stop-and-wait TCP loopback IPC on Port 5000 completed 200 round-trips in 306.92 ms (mean RTT = 1.535 ms, min = 0.311 ms, median p50 = 0.914 ms, p99 = 10.788 ms, ~652 msgs/sec—likely dominated by synchronous request-response framing and 512-character RAM hex string serialization on every step); (3) Latch-verified fault-isolation testing (Table 3 TR-07) proved that with the consumer blocked on a CountDownLatch and the 1,024-slot queue saturated (480 drops), the CPU executed Preset 1 (6x7=42) to HALT without stalling.

3. Simulation Examples Testing and Screenshots

Automated test suites (38/38 passing assertions across TestCPU: 20/20, TestCRC24b: 9/9, QueueTest: 3/3, and LoggerProcessTest: 6/6) validated all instruction categories, 20-item FIFO wrap-around, latch-verified IPC fault isolation, and exhaustive CRC-24b error detection (Table 3, Figures 2 and 3):

Table 3: Automated Verification Matrix With Explicit Source Test Suite Mapping (38/38 Passing)

Row	Operation & Source Test ID	Expected Outcome	Actual Outcome	Status
TR-01	Data Transfer: MOV A,#42H; B,#07H (CPU TC-01)	A=42H, B=07H, PC advances	A=42H, B=07H; PC incremented	PASS
TR-02	ALU Add/Borrow: ADD & SUBB (CPU TC-04, TC-18)	ADD: 01H,CY=1; SUBB: FDH,CY=1	A=01H (CY=1,AC=1); SUBB A=FDH (CY=1)	PASS
TR-03	Control Flow: SJMP, JZ/JNZ, 80 FE (TC-17, TC-19)	JZ/JNZ skip INC; 80 FE halts	INC skipped (A=00H); 80 FE sets isHalt	PASS
TR-04	Stack PUSH ACC (0xC0 0xE0) (CPU TC-15)	SP 07H->08H; RAM[08H]=3CH	SP=08H, RAM[08H]=3CH	PASS
TR-05	FIFO 20-Item Wrap-Around (CPU TC-20, QueueTest)	Fill 16, pop 4, push 4 wraps rear	full=true, wrapped 17..20, dequeued 16	PASS
TR-06	MUL AB (6x7) & DIV AB (B=0) (CPU TC-16)	A=2AH, OV=0; DIV B=0->OV=1	A=2AH, OV=false; DIV0 sets OV=true	PASS
TR-07	Latch-Blocked Logger CPU Step (LogTest TC-06)	CPU halts while consumer blocked	ACC=2AH, halted=true (480 queue drops)	PASS
TR-08	Exhaustive CRC-24b Sweep (TestCRC24b TC-01..09)	'123456789'->23EF52; all 1b/2b!=0	96/96 1-bit & 4,560/4,560 2-bit detected	PASS

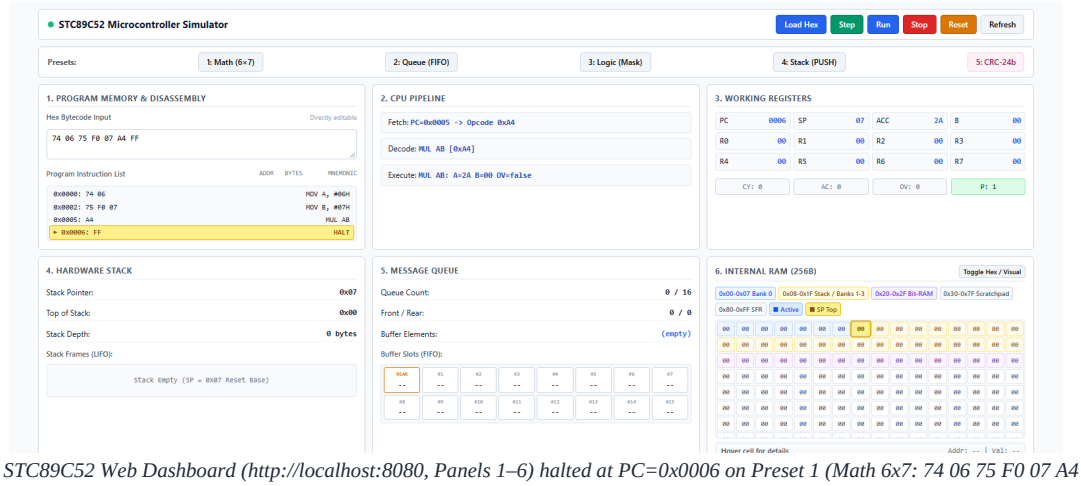


Figure 2: Live STC89C52 Web Dashboard (http://localhost:8080, Panels 1–6) halted at PC=0x0006 on Preset 1 (Math 6x7: 74 06 75 F0 07 A4 FF), showing ACC=2A (42), B=00, P=1, Pipeline MUL AB [0xA4], and 256B Internal RAM.

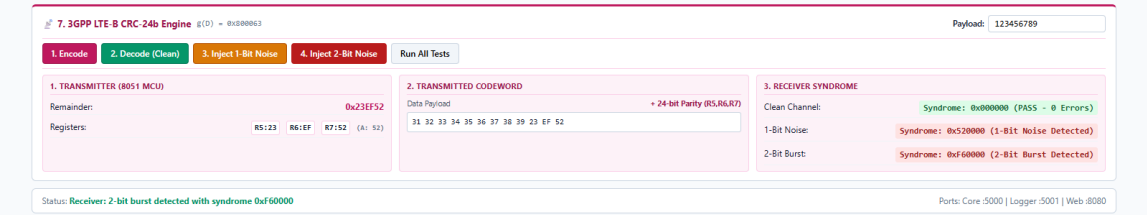


Figure 3: Cropped Panel 7 — 3GPP LTE-B CRC-24b Lab on Web Dashboard verifying "123456789" -> Remainder 0x23EF52 (R5:23 R6:EF R7:52), Clean Syndrome 0x000000, 1-Bit Error (Byte 0 Bit 0) 0x520000, and 2-Bit Burst (Byte 0 Bits 0&1) 0xF60000.

4. Individual Contribution

4.1 Weekly Task Breakdown by Member

Table 4: *Weekly Task Allocation Across All Four Team Members*

Sprint Week	Cyrus Shobith Pinto (Team Lead)	Shashidhara Poojari (UI & Visuals)	Chrisel Lobo (Core & FDE)	Arnold Noah Karyappa (Logger & Telemetry)
Week 1	Project setup, architecture, Git repo, task allocation	Web tech research, HTML layout wireframes	8051 register & memory mapping research	Logger requirements & file appender design
Week 2	Opcode specification, ALU design, build tooling	CLI terminal display & status monitor prototype	CPU FDE cycle engine, arithmetic/logic opcodes	Log format design & log category structure
Week 3	Data structure specifications, assembly test plan	Stack & Circular FIFO visualizer UI layout	DataStructures.java: SP stack & 16-slot queue	Thread-safe synchronized writeLog implementation
Week 4	Multi-process socket IPC & non-blocking queue	UIProcess.java HTTP web server & preset buttons	CoreProcess.java socket server & state serializer	LoggerProcess.java socket listener & file writer
Week 5	CRC-24b step8051Hardware(), benchmarks, report	3-Stage CRC-24b Lab UI & real-time noise controls	Registers R5:R6:R7 mapping, core report sections	LoggerProcessTest.java latch test & log audit

.

4.2 Agreed Team Contribution Summary

Table 5: *Agreed Team Member Workstream Distribution and Sign-Off*

Member Name	Student ID / USN	Assigned Workstreams	Confirmed Share (%)	Member Sign-Off
Cyrus Shobith Pinto	25190111	Architecture, Socket IPC, CRC-24b Engine, Benchmarking, Integration	25%	_____
Shashidhara Poojari	25190148	UIProcess HTTP Server, Responsive Dashboard, Visualizers, Presets	25%	_____
Chrisel Lobo	25190110	CoreProcess CPU Engine, ALU Opcodes, Registers, RAM, Stack, Docs	25%	_____
Arnold Noah Karyappa	25190106	LoggerProcess Daemon, LogHandler, File Persistence, Backpressure Tests	25%	_____

4.3 Individual Personal Reflections

Reflection by Cyrus Shobith Pinto (Team Leader - 25190111)

Early in the project, I underestimated how easily a slow socket reader could stall the entire simulator if CoreProcess wrote synchronously to LoggerProcess, and I initially conflated 32-bit Java integer shifts with true 8051 assembly execution. Re-engineering the logger path around a bounded 1,024-slot queue that drops excess frames rather than blocking the CPU—and explicitly modeling the 3GPP CRC-24b shift across R5:R6:R7—taught me how to make and document explicit systems trade-offs.

Reflection by Shashidhara Poojari (UI Subsystem Lead - 25190148)

When I first wired the browser disassembly table to CoreProcess, spaces in hex bytecode strings silently broke URL parsing and caused the UI to desynchronize from the Program Counter. Tracking down that bug to missing URL decoding on /api/cmd taught me that client-server interfaces fail at serialization boundaries, motivating me to keep the 7-panel layout and CRC noise controls simple and deterministic.

Reflection by Chrisel Lobo (Core Process Lead - 25190110)

My biggest surprise while coding the ALU was seeing SUBB and ADD produce wrong carry and borrow flags because Java automatically sign-extends signed bytes (turning 0xFE into -2). Fixing that bug with explicit & 0xFF masks across every register and memory read—and handling DIV AB divide-by-zero by setting OV=1 instead of crashing—showed me how unforgiving hardware emulation is when host-language types leak through.

Reflection by Arnold Noah Karyappa (Logging Subsystem Lead - 25190106)

I initially assumed opening a ServerSocket on Port 5001 and writing lines to simulator.log would be trivial, until concurrent test bursts risked interleaving log output and filling the buffer. Adding synchronized file appends in LogHandler and writing the CountDownLatch backpressure test taught me the trade-off between strict audit completeness and non-blocking producer performance.

5. Team Communication

Collaboration was maintained strictly using standardized agile and GitHub development practices:

- GitHub Pull Requests (PR #1 integration review) and branch reconciliation before merging feature branches (`week-01` through `week-05`, `chrisel-CORE`, `arnold-LOG`, `Shashidhara-UI-Final`, `cyrus-integration`) into `main`.
- Detailed weekly meeting minutes following standard templates (agenda, technical decisions, participant action items) recorded in `docs/meetings/`.
- GitHub Issues and automated test suites (`TestCPU`, `TestCRC24b`, `QueueTest`, `LoggerProcessTest`) used to track task assignments, bug reports, and architecture decisions (`docs/decisions/decision-log.md`).

6. Problems Faced and How They Were Resolved

Table 6: *Engineering Issues Encountered, Root Causes, and Implemented Resolutions*

Issue Reference	Category	Symptom / Design Hazard	Root Cause	Implemented Resolution
ISSUE-01	Java Datatypes	Signed integer sign-extension corrupted ADD/SUBB flags	Java lacks unsigned byte (0xFE becomes -2)	Applied bitwise mask (& 0xFF) across all ALU operations and register/RAM reads.
ISSUE-02	Silicon Memory	Stack underflow & stale ROM bytes on reload	SP reset base at 07H [2]; loadProgram needed clear	Guarded pop() at SP <= 0x07 (fix(stack) commit) and zero-filled codeMemory on load.
ISSUE-03	Concurrency	Unbounded/blocking socket writes can stall CPU if Logger freezes	OS TCP send buffer fills when reader stalls	Bounded ArrayBlockingQueue(1024) with offer() & drop counter (prioritizing CPU determinism).
ISSUE-04	Algorithm	24-bit 3GPP LTE CRC remainder exceeds 8-bit register width	STC89C52 registers are only 8 bits wide	Java routine step8051Hardware() shifts through carry bit across R5:R6:R7 in GF(2) XOR.
ISSUE-05	Fault Handling	DIV AB threw ArithmeticException on divide by zero	Unhandled software division by B=0	Implemented 8051 hardware spec [2]: B=0 sets Overflow (OV=1) without crashing.
ISSUE-06	Web Protocol	Hex bytecode with spaces failed to load from query string	HTTP URL encoding of spaces and symbols	Integrated URLDecoder.decode() in UIProcess /api/cmd HTTP request handler.

7. From Theory to Practice

7.1 Operating Systems Concepts

The project applies operating systems principles through process creation, inter-process communication (IPC) via loopback sockets, and bounded producer-consumer buffering. Decoupling the system into discrete OS processes (CoreProcess, LoggerProcess, UIProcess) with thread-safe monitor locks (cpuLock, synchronized LogHandler.writeLog) and a non-blocking 1,024-slot ArrayBlockingQueue ensures that a crash or stall in the UI or disk logger cannot block CPU instruction execution.

7.2 Data Structures & Algorithms

Data structures were implemented to model embedded execution buffers: a 16-slot circular FIFO ring buffer with modulo wrap-around pointers ((rear + 1) % 16) verified across 20-item wrap-around bursts, a Last-In First-Out (LIFO) hardware stack starting at SP=07H [2], and GF(2) linear feedback shift register polynomial division across R5:R6:R7 [5].

7.3 Computer Networks

Computer networking principles govern both the internal socket IPC wire framing and the telecommunications verification lab. The 3GPP TS 36.212 CRC-24b polynomial (g(D) = 0x800063) [5] demonstrates transport block error detection over noisy channels: for payload "123456789" (12-byte codeword 31..39 23 EF 52), clean reception yields syndrome 0x000000, whereas an exhaustive test sweep verified that all 96 single-bit errors (e.g., byte 0 bit 0 -> 0x520000) and all 4,560 two-bit error pairs (e.g., byte 0 bits 0&1 -> 0xF60000) produce non-zero syndromes.

7.4 Cross-Disciplinary Synthesis Reflection

Developing this simulator demonstrated how core computer science disciplines intersect in systems software [4]. Effective virtualization requires balancing computer architecture (instruction-accurate state machines [2]) with operating systems principles (isolated processes, bounded non-blocking queues, synchronization) and networking techniques (TCP framing and GF(2) cyclic error detection [5]).

8. Conclusion, Limitations and Future Work

This project engineered, integrated, and verified an educational multi-process simulator for a 19-opcode subset of the STC89C52 (8051) microcontroller [2]. The system delivers an instruction-accurate Fetch-Decode-Execute CPU engine (supporting both synthetic 0xFF HALT and standard 8051 SJMP $ [80 FE] halting), 8 KB ROM, 256-byte RAM, a 16-slot circular FIFO queue, an asynchronous append-only TCP logger on Port 5001, a 7-panel Web Dashboard on Port 8080, and a 3GPP TS 36.212 CRC-24b lab [5], passing all 38/38 automated test assertions.

System Scope, Trade-offs & Limitations: (1) The CPU implements 19 representative opcodes (~7.5% of the 255-opcode MCS-51 ISA) and assigns custom educational opcodes 0xE0/0xD0 to FIFO enqueue/dequeue and 0xFF to synthetic HALT, which collide with standard 8051 MOVX A,@DPTR, POP direct, and MOV R7,A; (2) CRC-24b (step8051Hardware) is implemented as a Java routine manipulating the simulated register file (R5:R6:R7 and CY) rather than fetched 8051 machine bytecode; (3) the bounded 1,024-slot log queue prioritizes CPU determinism over log completeness by dropping frames if LoggerProcess stalls; and (4) hardware timers and UART interrupts are not yet clocked.

Future Work: Remap the custom FIFO queue and synthetic halt hooks to memory-mapped Special Function Registers (SFRs) or reserved opcode 0xA5, expand the opcode table to all 255 MCS-51 instructions with machine-cycle timing (e.g., 4-cycle MUL/DIV) to run compiled Keil/SDCC Intel HEX binaries, implement CRC-24b in native 8051 assembly, and add CH340 USB-UART hardware-in-the-loop bridging.

References

[1] Department of Computer Science & Engineering, "PBL Simulator Evaluation Rubrics & Task Sheets," St. Aloysius (Deemed to be University), Mangaluru, 2026.

[2] Intel Corporation, "MCS-51 Microcontroller Family User's Manual," Publication No. 272383-002, 1994; and STC Micro, "STC89C52RC/54RD+ Series 8-Bit Microcontroller Datasheet," 2011.

[3] P. C. Vijay Ganesh, "8051 Microcontroller Web-based Simulator," 2021; and J. Rogers, "EdSim51 — The 8051 Simulator for Teachers and Students," Virtualidian, 2019.

[4] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed. Morgan Kaufmann, 2014.

[5] 3GPP TS 36.212, "Evolved Universal Terrestrial Radio Access (E-UTRA); Multiplexing and channel coding (Release 15)," Section 5.1.1 (CRC-24b generator polynomial gCRC24B(D)), 2018.

Final Report Sign-off

By signing below, the team members confirm that the documented contribution distributions, source code statistics, and peer evaluations represent an accurate record of the project lifecycle.

Team Leader Signature: _____ Date: 07 November 2026

Faculty Mentor Endorsement:

I have reviewed this Project-Based Learning Final Report along with the associated repository, code contributions, and test suite.

Mentor Signature: _____ Date: _____